Unified Morphing Architectures: Revised Draft

Nov 3, 2026 · @G

Abstract

Unified Morphing Architectures (UMA) replace the stored program with a learned synthesiser that emits short, bounded instruction sequences on demand from the machine's current state. Each sequence passes a checker, runs in a fast burst on a RISC-V core with reconfigurable logic, and is then discarded. A recorded trace of what was emitted makes every run auditable and replayable.

UMA is best understood as an interpreter or JIT whose front end is a learned model instead of a bytecode stream. It does not aim to beat conventional execution on throughput. It proposes a different computational medium, in which the boundary between hardware and software is dissolved at the edges while proven paths harden into ordinary fast machinery. The same substrate can serve as a network switch, a phone or any other gadget, differing only in intent, peripherals and invariants, and units can be composed into coordinated hierarchies.

1. Introduction

Since the Von Neumann architecture, computing has run on a script-and-reader model. Software is built, frozen into a binary, loaded into memory, and fetched and executed in order. The costs are familiar: storage and memory-wall latency, exposure to static-analysis attacks, and algorithms bent to fit fixed silicon.

UMA asks what changes if instructions are generated at the moment they are needed, from the live state of the machine, rather than read from a stored program. Instruction streams become transient. They are produced, checked, executed and, if wanted, logged, but they are not authored in advance and shipped as files.

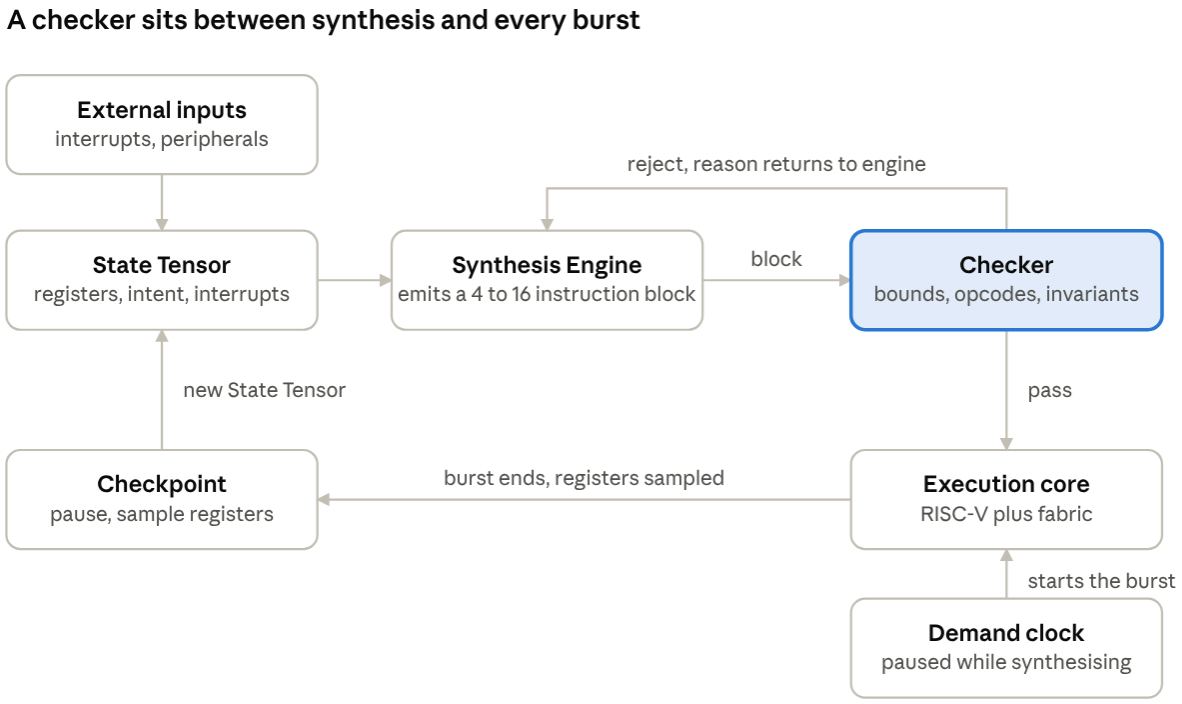
What this paper claims. The hardware/software boundary can be dissolved at the point where instructions are produced, and the resulting system is observable and reproducible because the emitted stream can be tapped, logged and replayed.

What it does not claim. That information persisting in the machine disappears. Whatever shapes the synthesiser's behaviour (parameters, fabric configuration, analog state) is the program's content in another medium. UMA changes where that content lives and when

instructions are formed, not whether the machine has behaviour that was specified somewhere. It also does not claim a throughput advantage over compiled code.

2. Architecture

Four components replace the conventional fetch-decode-execute control loop.



architecture loop · 6 components, 1 reject loop

A block must pass the Checker before the demand clock starts a burst, and the checkpoint feeds the next State Tensor.

State Tensor. In place of a program counter pointing into memory, execution state is a vector holding the architectural registers, a representation of current intent, and the live interrupt lines. It is the synthesiser's only input. Within a burst a small sequential index steps through the emitted instructions, so a counter still exists, but it indexes a block, not a program.

Synthesis Engine. A dedicated accelerator runs a model over a constrained token space of machine opcodes and operands. It predicts the next short block that moves the machine toward the current intent. How the model is realised (fixed weights in silicon, adaptive, analog, or embodied in the fabric) is left open by the architecture. UMA only requires that it consume a State Tensor and emit a bounded block.

Checker. Between synthesis and execution sits a verifier that accepts or rejects each block before the clock fires. It enforces memory bounds, permitted opcodes and fabric configurations, and task-specific invariants. This is the choke point that makes small,

bounded generation safe and not merely small. Because a block is short and loop-free, it can be verified as deterministic by a mathematical checker, akin to an SMT solver or proof checker and possibly AI-assisted, which confirms that the block's effect on the State Tensor matches the intended transition before it runs. A rejected block returns to the engine with the reason as additional input.

Demand Clock. Execution is not driven by a fixed oscillator. The core rests in a paused, low-power state while the engine synthesises, and a hardware flag raises the clock edge when a checked block is ready. The machine therefore advances at the pace of synthesis plus checking, and stalls during that time by design, not by accident.

System	Clock	Program state	Execution style
Single-core	One fixed heartbeat	One program counter	Linear, one step at a time
Multi-core	One master clock, independent throttles	Several program counters	Parallel, simultaneous steps
UMA	Demand-driven asynchronous pulses	State Tensor plus in-burst index	Checked micro-bursts

3. Micro-Burst Execution

The engine never predicts a whole application. It emits a finite block of typically 4 to 16 instructions, and the machine cycles through four phases.

1. **Pulse.** The engine reads the State Tensor and emits a block.
2. **Check.** The Checker validates the block against bounds, opcode and fabric rules, and invariants. A failure loops back to the Pulse phase.
3. **Burst.** The clock runs at full speed through the block with a simple sequential index.
4. **Checkpoint.** The clock pauses, registers are sampled into a fresh State Tensor, and any interrupt or external change is folded into the next Pulse.

Reproducibility. Generation may be stochastic, but execution is reproducible because the emitted block is a concrete instruction sequence that can be recorded. A trace of blocks together with the logged external inputs (interrupt vectors, peripheral and network data) that shaped each State Tensor is enough to replay a run exactly. The same tap supports logging, auditing, positive and negative feedback for training, and fault analysis, as in record-and-replay debuggers and tracing JITs.

Consequence. A persisted trace is a recording of what the machine did, which is itself a software artifact. UMA's position is that code need not exist as an authored file before execution. It can exist afterwards, as a record.

4. The Ephemeral ISA

The execution fabric combines an open RISC-V base with reconfigurable logic.

Baked-in primitives. A hardwired core of standard RISC-V instructions (ADD, SUB, AND, OR and the rest of the base set) handles fundamental arithmetic at full efficiency with no routing overhead.

Custom extensions. For a hot or awkward operation, such as an unaligned multi-stage shift-and-accumulate, the engine can select a custom instruction in the RISC-V extension space instead of emitting several base instructions. The paper's original mechanism had the engine emit a micro-bitstream during the clock pause to wire a fabric block, minting an instruction such as DYN_MACRO_01 for one burst.

Reconfiguration cost. Partial reconfiguration of real FPGA fabric takes far longer than a 4 to 16 instruction burst, so rewiring per burst only pays off when the custom op is reused many times. Three refinements address this:

- **Pre-staged variants.** Candidate custom ops are held in neighbouring fabric regions, and "minting" an instruction means selecting one, a mux change, not reprogramming gates.
- **Coarse-grained reconfiguration.** On coarse-grained reconfigurable arrays a context switch costs cycles, not a bitstream load.
- **Promotion by use.** A custom op that keeps recurring is made permanent. Dissolving back to neutral fabric applies only to rare ones.

Checked configurations. Any fabric configuration the engine proposes is validated by the Checker before it is applied. A malformed bitstream can create contention and damage hardware, so configurations are drawn from a vetted, parameterised set and not generated freely.

5. Interrupts and Asynchronous Events

Because execution proceeds in bounded bursts, an interrupt does not need to preempt mid-instruction. It alters the State Tensor, the current burst finishes, and at the next checkpoint the engine reads the interrupt vector as a high-priority input and synthesises a handler block. The prior intent path stays resident in the State Tensor's saved registers until the engine routes back to it.

Latency. Interrupt latency is the remaining burst plus a synthesis and check pass. That is acceptable for many workloads and unacceptable for hard real-time ones. The architecture therefore needs a small fixed fast path for urgent events, with synthesis reserved for the rest.

State saving. Keeping the old state in inactive registers is still a form of context save. UMA simplifies it by tying it to burst boundaries, not removing it.

6. Optimisation Trajectory

The costs identified above are engineering problems that known techniques erode, and each points the same way: hot paths harden, novelty stays synthesised.

Cost	Mitigation	Effect
Synthesis latency	Cache frequently emitted blocks, keyed on a quantised State Tensor; synthesise only on a miss	Hot path approaches conventional hardware speed
Synthesis latency	Speculatively generate the next block while the current one executes	Demand clock waits less
Reconfiguration time	Pre-staged variants, coarse-grained arrays	Minting an op costs cycles, not a bitstream load
Recurring custom ops	Promote to permanent hardware by usage	Dissolving applies only to rare ops
Interrupt latency	Fixed fast path for urgent events; handlers as cacheable blocks	Real-time cases served without synthesis

Where this lands. A trace cache with a learned front end. The ephemeral path is the cold path, handling novelty, adaptation and rare cases. Proven paths harden into ordinary machinery. This is not a retreat from the idea. It is the stable form of it, in which the dissolution of the hardware/software boundary is real at the edges and the core is fast and checkable.

7. Simulation Findings

Cache hit rate and checker cost per block decide whether UMA runs near ordinary hardware speed or orders of magnitude slower. A small simulator (`uma_sim.py`, stdlib Python) models the pulse, check, burst and checkpoint loop, with the synthesiser, checker and fabric as black boxes. The synthesiser is an oracle with injected errors. The

checker shadow-executes each block against the intended transition and applies static safety rules. The workload is 24 loop kernels with Zipf popularity over 2000 tasks. All timings are placeholder assumptions, so the results show shape, not absolute performance.

Question	Result
What does the trace cache buy?	Slowdown against a conventional core falls from 226 to 788 times with no cache to about 3 times with 1024 entries, at 98 to 100% hit rate
Does burst size matter once cached?	Larger bursts cost more, because the hit rate falls (100% at burst 4, 98% at burst 32; 3.3 times to 4.6 times)
Does speculation help?	Only when synthesis plus check takes about one burst time (18 ns) or less. At 4,120 ns it changes nothing; at 16 ns it hides the pulse (2.17 times to 1.18 times)
What does the checker buy?	Full coverage gave zero wrong blocks and zero safety incidents up to 5% synthesiser error. At 90% coverage 58 wrong blocks got through; with safety rules only, 551
What does a cache do to checker escapes?	It cuts the count but makes survivors persistent: at 20% error and 50% coverage, 589 of 596 wrong blocks were replays of cached ones
Fabric strategy for a custom op?	Rewiring per burst took 729 ms against 1.76 ms with no custom op. Promotion was fastest (1.61 ms); pre-staging narrowly lost (1.80 ms) because its mux switch costs more than the 3 ns per element it saves
Interrupt latency?	Dominated by synthesis already in flight, not the remaining burst. With a cached handler, p50 was 0.012 µs at burst 4 against 4.4 µs uncached

Implications. The block working set must fit the cache, or the architecture is not viable on these assumptions. Because a cache turns an undetected error into a repeated one, only blocks that passed the full check should enter it. A fast path for urgent interrupts must be able to abort synthesis, which the simulator does not yet model.

Limits. The synthesiser is an oracle, not a learned model. The checker compares against a known-good block, so the simulator says nothing about how hard it is to write the specification a real checker needs. Cached blocks are assumed reusable across data. Interrupt percentiles for cached runs rest on about 100 samples.

8. No Application, No Operating System

In a conventional device the application is the stored program of intent, and the operating system is the stored program of resource policy: scheduling, isolation, drivers, files. UMA has neither as a stored artifact. What remains is a substrate that is the same on a network switch, a phone or a small gadget. Devices differ in three things only: their intent, their peripherals, and their invariants.

Conventional role	UMA counterpart
Application	Intent held in the State Tensor, refreshed from inputs. A switch's intent is to forward frames per learned topology and policy; a gadget's is to sample, filter and report
Scheduler	Demand clock and synthesiser, which emit whatever the moment needs
Device drivers	Synthesised peripheral-handling blocks, confined to ranges the Checker approves
Memory protection and isolation	Checker rules on what any block may read, write or reconfigure
Logging, accounting, files	The trace and the input log
Install, update	A change of intent, invariants or engine parameters, not a new binary

What does not disappear. Persistent policy has to live somewhere, and in UMA it lives in the Checker's invariants and in whatever shapes the engine. These are the residual operating system. The design aim is to concentrate all persistent, trusted policy in the Checker, which is small enough to verify, and keep everything else ephemeral.

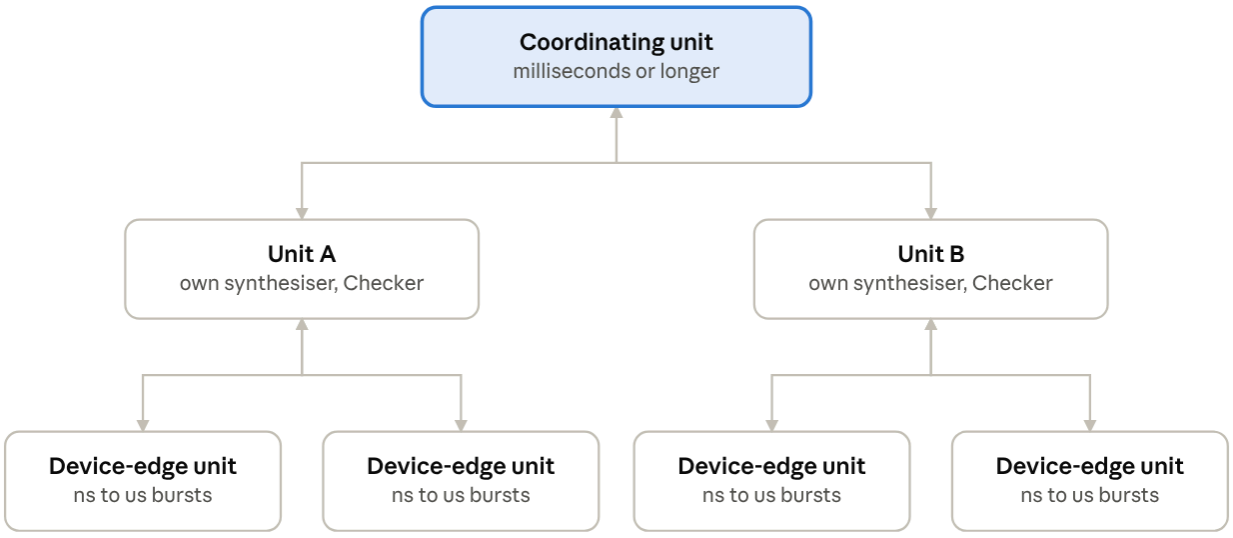
Authorship moves, it does not vanish. Intent and invariants are still specifications, and someone or something must write them. UMA moves authorship from instructions to goals and constraints. How those are expressed, and how they are updated on a deployed device, is the central open design question.

Boot and trust. Something must establish the initial intent and invariants and protect them from tampering. That root of trust is a small fixed element, which is acceptable as long as the paper says so.

9. Coordinated Hierarchies

UMA units compose. One unit's intent can be supplied by another unit, so a device can be a hierarchy of units, each with its own synthesiser and Checker. Intent flows down and state and traces flow up. Each level runs at its own timescale: device-edge units burst in nanoseconds to microseconds, coordinating units deliberate over milliseconds or longer.

Intent flows down, state flows up, and every unit checks



Arrows run both ways: intent down, checked on receipt; state and traces up.

hierarchy · 7 units, 3 levels

Each unit accepts intent from above only after its own Checker approves it, and reports state and traces back up.

Bounded delegation. A unit checks incoming intent against its own invariants before accepting it. A wrong or compromised parent therefore cannot push a child outside the child's invariant set. Safety is local and composes, even though correctness of the overall behaviour does not.

Coordination by goals and state, not code. Peers and parents exchange intent and state summaries, never instruction streams. There is nothing to deploy: a unit is re-purposed by changing what it is asked to do. Incoming messages are data, validated on receipt like any other input.

Fast and slow paths. Reflexes stay local, as with the interrupt fast path. Slower, more expensive synthesis happens higher up, where its latency is tolerable.

What this does not solve. Hierarchies bring distributed-systems problems: consistency, delay, feedback stability, and untrusted peers. Checkers at each level bound the damage but do not guarantee liveness or good coordination. How intent summaries compress as they pass upward, and how mixed hierarchies containing conventional nodes behave, are open.

10. Security and Safety Model

Removing the stored binary removes one class of attack, tampering with a file at rest, and changes where the others sit.

- **Input as attack surface.** Peripheral and network data shape the State Tensor, so crafted inputs can steer synthesis in ways analogous to prompt injection. Inputs need validation and the Checker must not trust the engine's intent.
- **Auditability by trace.** The recorded block stream and input log make behaviour inspectable after the fact. This restores the audit trail that the absence of a static binary would otherwise remove.
- **Bounded blast radius.** A 4 to 16 instruction block can still perform a stray store or a bad GPIO write, so bounds, permitted opcodes and write-region limits are enforced by the Checker, not left to the engine.
- **Model protection.** Whatever holds the engine's behaviour is a high-value target and needs the same integrity and extraction protections as firmware.

11. Related Work, Open Questions and Conclusion

Related work. UMA sits near several established areas: JIT and tracing compilers, dynamic binary translation and code-morphing processors, record-and-replay systems, coarse-grained reconfigurable arrays and partial FPGA reconfiguration, self-timed and clockless logic, and program synthesis. For the no-operating-system view, relevant work includes unikernels, library operating systems, exokernels and capability hardware such as CHERI. For hierarchies, it includes layered control architectures such as subsumption and multi-agent control hierarchies. Its distinguishing element is a learned synthesiser as the instruction source, checked at every burst. A fuller treatment should position UMA against each.

Open questions.

- How is intent expressed and encoded in the State Tensor, and what language do the Checker's invariants use? This is the specification the Checker verifies against.

- Which real workloads have block working sets that fit a cache? The simulation shows hit rate and checker cost per block are decisive.
- What is the right burst size for a learned model? The simulation shows larger bursts cost more once cached, but that was with an oracle synthesiser.
- How is the engine trained, and how do trace and feedback loops update it without breaking reproducibility?
- How is cache admission restricted to fully verified blocks, and how are bad cached blocks found and evicted?
- How can an interrupt fast path abort in-flight synthesis, and what latency does that reach?
- In hierarchies, how do intent summaries compress as they pass upward, how stable are feedback loops across levels, and how do mixed hierarchies with conventional nodes behave?
- What root of trust fixes the initial intent and invariants, and how are they updated on a deployed device?

Conclusion. UMA is a proposal for a machine whose instructions are formed at the moment of need, checked, executed and recorded, not authored and stored. It does not abolish software so much as move its content into the synthesiser, its record into the trace, and its authorship from instructions to intent and invariants. The same substrate can be a switch, a phone or any gadget, and units can be composed into hierarchies that coordinate by goals and state, not code. A first simulator (section 7) shows that cache hit rate and checker cost per block decide viability. The next steps are to specify the Checker's invariant language, replace the oracle synthesiser with a real model, and simulate a small hierarchy of units.